

Week 7 Assignment

Report Delta Lake — Incremental Load & Slowly Changing Dimension (SCD) using MERGE

Objective

The objective of this assignment is to implement an incremental data processing pipeline using **Delta Lake** and **PySpark**. The project demonstrates how to load customer data into a Delta table, clean the data by removing null values and duplicates, perform incremental updates using the **MERGE** operation, validate the results, and display the final updated dataset.

Tools and Technologies Used

- Microsoft Azure Databricks
- PySpark
- Delta Lake
- Python
- CSV Files
- Unity Catalog

assignmnet7/

```
|— data/
|   |— customer_master.csv
|   └— customer_incremental.csv
|— notebooks/
|   └— delta_scd.ipynb
|— screenshots/
|   |— data_loading.png
|   |— data_cleaning.png
|   └— merge_operation.png
```

```
|   |— validation.png
|   |— final_output.png
|— report/
|   |— Delta_Lake_Assignment_Report.pdf
|— README.md
```

Dataset Description

Customer Master Dataset

The master dataset contains customer details such as:

- Customer ID
- Customer Name
- City
- Email Address

It intentionally contains:

- One duplicate customer record
- One record with a null customer name

Example:

Customer ID	Name	City	Email
101	Rahul Sharma	Delhi	rahul@gmail.com
102	Priya Verma	Mumbai	priya@gmail.com
103	Amit Patel	Ahmedabad	amit@gmail.com
104	Sneha Reddy	Hyderabad	sneha@gmail.com
105	Arjun Singh	Jaipur	arjun@gmail.com
106	Neha Gupta	Pune	neha@gmail.com
106	Neha Gupta	Pune	neha@gmail.com
107	NULL	Bhopal	abc@gmail.com

Incremental Dataset

The incremental dataset contains updated and new customer records.

Customer ID	Name	City
103	Amit Patel	Surat
105	Arjun Singh	Udaipur
108	Karan Mehta	Indore
109	Ananya Joshi	Nagpur

Changes made:

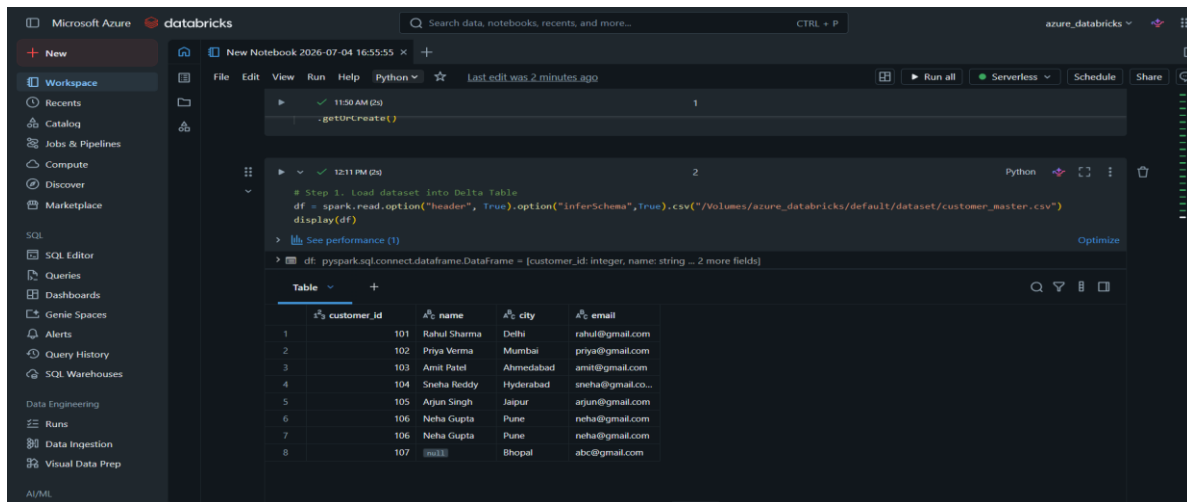
- Customer 103 city updated from Ahmedabad to Surat.
- Customer 105 city updated from Jaipur to Udaipur.
- Customer 108 inserted as a new customer.
- Customer 109 inserted as a new customer.

Methodology

Step 1 – Load Dataset

The customer master CSV file was uploaded into Azure Databricks and loaded using PySpark.

The dataset was displayed to verify that all records were loaded successfully.

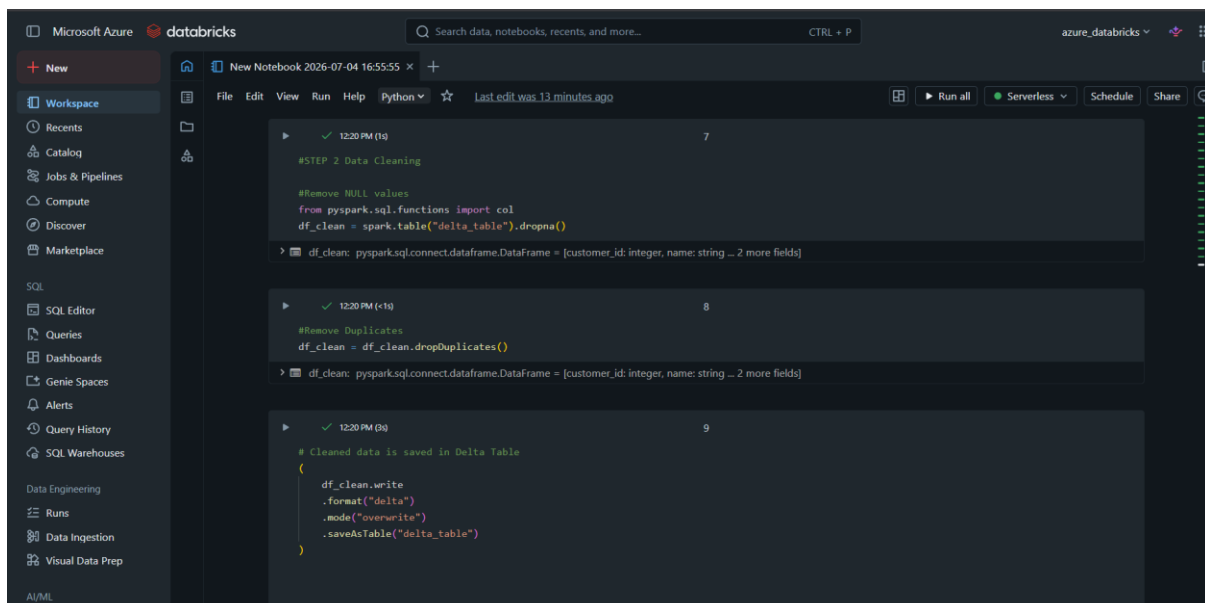


Step 2 – Data Cleaning

The following cleaning operations were performed:

- Removed records containing null values.
- Removed duplicate customer records using the customer_id column.

This ensured that only valid and unique records were stored in the Delta table.



Step 3 – Store Data as Delta Table

The cleaned dataset was written into a Delta table using:

- Delta format
- Overwrite mode

This Delta table acts as the master table for future incremental processing.

Step 4 – Load Incremental Dataset

A second CSV file containing updated and new customer records was loaded into PySpark.

This dataset simulates incremental daily customer data.

Step 5 – MERGE Operation

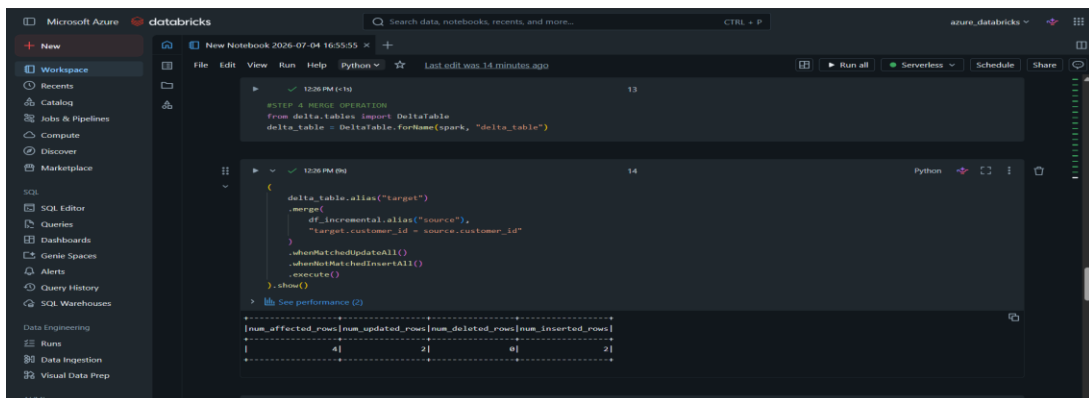
The Delta Lake MERGE command was used.

The operation performed:

- Update existing customer records when Customer ID already exists.
- Insert new customer records when Customer ID does not exist.

During execution:

- Updated Records = **2**
- Inserted Records = **2**
- Deleted Records = **0**



```
Microsoft Azure databricks
New Notebook 2026-07-04 16:55:55
File Edit View Run Help Python Last edit was 14 minutes ago
12:38 PM (14)
STEP 4: MERGE OPERATION
from delta.tables import DeltaTable
delta_table = DeltaTable.forName(spark, "delta_table")

12:38 PM (16) Python
{
  delta_table.alias("target")
    .merge(
      df_incremental.alias("source"),
      "target.customer_id = source.customer_id"
    )
    .whenNotMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
  }.show()
> See performance (2)
num_affected_rows|num_updated_rows|num_deleted_rows|num_inserted_rows|
-----|-----|-----|-----|
4|2|0|2|
```

Validation

After performing the MERGE operation, validation was carried out.

Validation checks included:

- Final row count
- Duplicate customer ID check

Results

Final Row Count

8

Duplicate Records

No duplicate customer_id values found.

This confirms that the MERGE operation was executed successfully.

```
1227 PM (15)
#STEP 5
#FINAL ROW COUNT
print("Final Row Count:", spark.table("delta_table").count())
Final Row Count: 8

1227 PM (16)
#DUPLICATES CHECK AFTER CLEANING
from pyspark.sql.functions import count

dup_df = spark.table("delta_table").groupBy("customer_id").count().filter("count > 1")
dup_count = dup_df.count()

if dup_count == 0:
    print("No duplicate customer_id values found.")
else:
    print(f"{dup_count} duplicate customer_id value(s) found.")
    dup_df.show()
dup_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]
No duplicate customer_id values found.
```

Final Dataset

The final Delta table contains eight customer records.

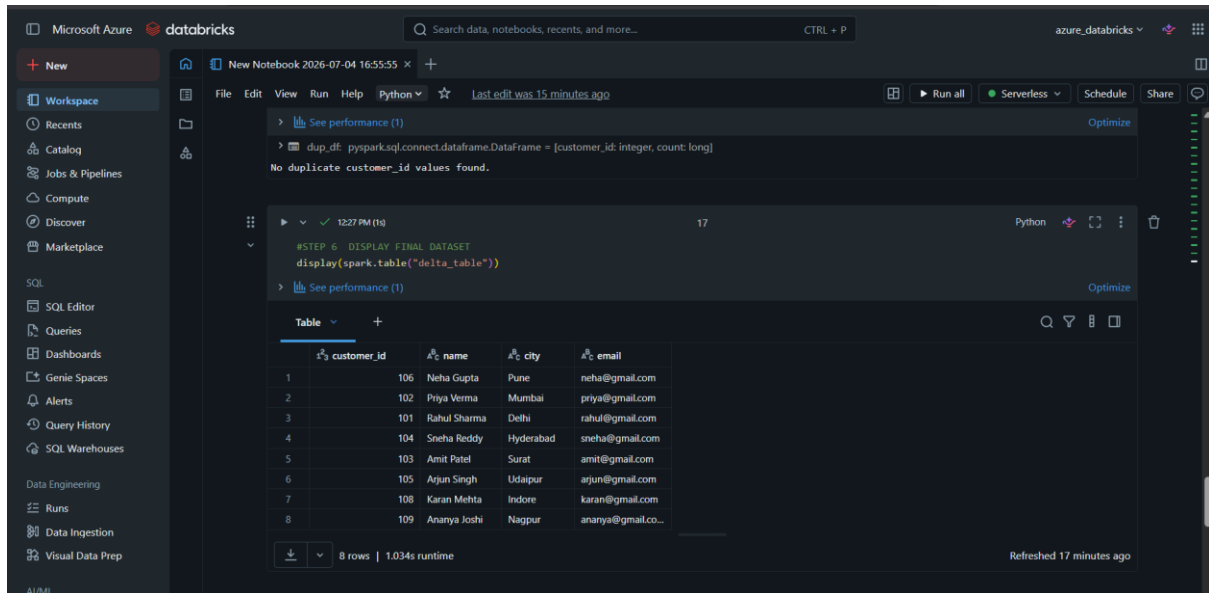
Updated customers:

- Amit Patel → Surat
- Arjun Singh → Udaipur

New customers inserted:

- Karan Mehta
- Ananya Joshi

The table contains no duplicate customer IDs and no null values.



The screenshot shows the Databricks interface with a notebook titled 'New Notebook 2026-07-04 16:55:55'. The notebook contains the following code:

```
> See performance (1)  
> dup_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]  
No duplicate customer_id values found.  
  
#STEP 6: DISPLAY FINAL DATASET  
display(spark.table("delta_table"))  
> See performance (1)
```

The table view displays 8 rows of data:

	customer_id	name	city	email
1	106	Neha Gupta	Pune	neha@gmail.com
2	102	Priya Verma	Mumbai	priya@gmail.com
3	101	Rahul Sharma	Delhi	rahul@gmail.com
4	104	Sneha Reddy	Hyderabad	sneha@gmail.com
5	103	Amit Patel	Surat	amit@gmail.com
6	105	Arjun Singh	Udaipur	arjun@gmail.com
7	108	Karan Mehta	Indore	karan@gmail.com
8	109	Ananya Joshi	Nagpur	ananya@gmail.co...

The table is refreshed 17 minutes ago and has a runtime of 1.034s.

Results

The implementation successfully demonstrated incremental data processing using Delta Lake.

The following tasks were completed successfully:

- Loaded the master dataset.
- Removed null values.
- Removed duplicate records.
- Created a Delta table.
- Loaded incremental customer data.
- Updated existing customer records.
- Inserted new customer records.
- Validated the final dataset.
- Displayed the updated Delta table.